Presenter script

Decentralised Single-Index Bandits

CS6007 pitch to Prof. Avishek Ghosh · **Harshitha** and **Nischal**

How to read this. Black text is what you say, written to be said aloud. Grey text is the part you drop when you have 15 minutes instead of 20. *[Blue brackets]* are things to do or point at. Each slide's header gives its time budget and the clock time at which you should click to the next slide. If you are more than 30 seconds behind, switch to the 15-minute plan from that slide on.

The speaker split is a suggestion: Harshitha takes the problem and the journey (Steps 1–4) and closes; Nischal takes the theory and the results. That gives each of you about 10 minutes.

#	Slide	Who	20 min	leave	15 min	leave
1	Title	H	15	0:15	15	0:15
2	Problem statement	H	55	1:10	50	1:05
3	Literature survey	H	50	2:00	35	1:40
4	Contributions	H	55	2:55	40	2:20
5	Recap: single-index bandits	H	45	3:40	45	3:05
6	With a server vs without	H	25	4:05	25	3:30
7	Our path in one picture	H	45	4:50	45	4:15
8	Experimental setup	H	25	5:15	skip	
9	Step 1: reproduce ZoomSIB	H	35	5:50	35	4:50
10	Step 2: add a server	H	55	6:45	45	5:35
11	Step 3: mix or relay?	H	50	7:35	40	6:15
12	Step 3, continued	H	45	8:20	skip	
13	Step 4: test each claim	H	45	9:05	45	7:00
14	Theory: setting	N	30	9:35	skip	
15	Theorem 5	N	45	10:20	35	7:35
16	Theorem 7	N	50	11:10	35	8:10
17	Step 5: first proof, epochs	N	50	12:00	50	9:00
18	Step 6: the algorithm	N	50	12:50	50	9:50
19	Theorem 27	N	50	13:40	50	10:40
20	Why Theorem 27 holds	N	65	14:45	50	11:30
21	Evidence: proved shape	N	40	15:25	40	12:10
22	Step 7: bin owners	N	50	16:15	50	13:00
23	Evidence: collaboration pays	N	20	16:35	skip	
24	Theorem 29: communication	N	30	17:05	30	13:30
25	Evidence: relaying vs mixing	N	25	17:30	skip	
26	Robustness	N	25	17:55	skip	
27	Theorem 33	N	25	18:20	skip	
28	Prior algorithms on our problem	N	30	18:50	30	14:00
29	What the failures taught us	H	25	19:15	25	14:25
30	Conclusion and future work	H	40	19:55	30	14:55
31	References	H	0		0	
32	Thank you	H	5	20:00	5	15:00

1 Title — Harshitha — 15 s, leave at 0:15

Good morning. I'm Harshitha, and this is Nischal. Our project asks what happens to single-index bandits when there is no server and the agents can only talk to their neighbours.

15 min: same.

2 Problem statement — Harshitha — 55 s, leave at 1:10

Here's the setting. N agents sit on a graph. Every round, each agent sees K contexts, pulls one, and gets a noisy reward: f of x dot theta-star. Theta-star and f are shared by everyone, both are unknown, and f may be non-monotone. We want small regret summed over the whole network, and we want to send little.

Four things make this hard. There are two unknowns: the direction needs data pooled from everyone, and the shape of f needs counts per bin. There's no coordinator, so every total has to travel across the graph and arrives about a diameter late. Stale or averaged counts break the usual UCB confidence argument; we'll come back to that. And sharing full tables every round is expensive.

15 min: 50 s, leave at 1:05. Drop the grey sentence.

3 Literature survey — Harshitha — 50 s, leave at 2:00

Dey, Bhore and Ghosh give the optimal T-to-the-two-thirds rate for one agent with a non-monotone f. That's our base algorithm. Kang et al. give ESTOR and GSTOR, also for one agent, and we use them as baselines. Wang et al. give us the event trigger, but they need a server. On the decentralised side, DDUCB and Landgren handle finite arms, DLUCB handles linear models, and Coop-KernelUCB handles kernels and leaves the graph lower bound open. *[point at the box]* Nobody learns an unknown index and an unknown non-monotone link over a graph, and that's where we work.

15 min: 35 s, leave at 1:40. Drop the grey sentences.

4 Contributions — Harshitha — 55 s, leave at 2:55

We have five contributions. First, the problem: as far as we know, this is the first decentralised single-index bandit. Second, lower bounds. No serverless algorithm beats NT to the two-thirds, and on rings and paths there's an extra term that grows with the diameter. Third, an upper bound for the algorithm we actually run: the server's rate plus a graph term that doesn't grow with T, for every trigger threshold gamma. Fourth, communication is logarithmic in T, and directed, delayed or changing graphs only change the delivery time. Fifth, 37 experiments, all with paired seeds and 95 percent confidence intervals. They show the graph cost has the shape the theorem predicts, and they compare us with prior algorithms on our problem.

15 min: 40 s, leave at 2:20. Drop the grey sentences.

5 Recap: single-index bandits — Harshitha — 45 s, leave at 3:40

Everything builds on ZoomSIB-UCB, so here's a quick recap. Phase one explores at random and estimates the direction with a Stein estimator. That estimate is a sample mean: a sum divided by a count. Phase two projects each arm onto the direction, cuts the line into bins, and runs sleeping UCB over the bins that have an arm this round. Its state is a count and a reward sum per bin. So both phases only keep sums. *[point at the red box]* Our question: N agents share theta-star and f, but there's no server.

15 min: same.

6 With a server vs without a server — Harshitha — 25 s, leave at 4:05

On the left, a server sees every statistic. On the right is our setting: a graph, where the diameter and the spectral gap decide how fast information moves. Think of sensor networks, drone swarms, or groups that won't hand raw data to a central party. Regret is summed over all agents and all rounds.

15 min: same.

7 Our path in one picture — Harshitha — 45 s, leave at 4:50

The rest of the talk follows this line. Each dot is a step, and the red text under it is what went wrong or what it cost. *[trace left to right]* Our pitch was to average estimates, and that turned out to be biased. Adding a server showed that model averaging fails. Removing it showed that mixing costs about two thousand times the messages. Our first proofs were either loose or slow, and each fix led to the next step. We end with a relay we can prove, plus a rule that removes most of what's left.

15 min: same.

8 Experimental setup — Harshitha — 25 s, leave at 5:15

We use Dey, Bhore and Ghosh's instances: d is 10, K is 20, with a quadratic, a zigzag and a logistic link. We have sixteen agents by default, on graphs from complete to path. Every method runs on the same seeds, and we report the mean with a 95 percent interval.

15 min: skip; click straight to slide 9.

9 Step 1: reproduce ZoomSIB with one agent — Harshitha — 35 s, leave at 5:50

Step one: we reproduced ZoomSIB alone. The Stein error falls at the expected one-over-root-n rate. ZoomSIB beats LinUCB, ESTOR and GSTOR on the non-monotone links, and phase one needs only about 2 percent of the horizon. *[point at the red box]* What we took from it: the whole learning state is sums, and sums add exactly. Our pitch had planned to average each agent's estimate, and we dropped that idea here.

15 min: same.

10 Step 2: add a server, and our first plan fails — Harshitha — 55 s, leave at 6:45

Step two: add a server. This is where our pitch plan failed. Averaging each agent's normalised direction gave an error of 0.33, against 0.20 for pooling the Stein sums first. For the bin tables we tried federated model averaging. After syncing, the table was 5 to 20 times further from the truth than if we hadn't synced at all, because FedAvg weights an agent by its total data and ignores how often it pulled that bin. So we send exact counts and sums. Syncing every round cost one and a half million scalars; an event trigger with gamma 0.5 got the same regret with thirty thousand. *[point at the bottom box]* With a server, collaboration pays: regret grows like N to the 0.48, against linear for independent agents.

15 min: 45 s, leave at 5:35. Drop the grey sentences.

11 Step 3: remove the server: mix or relay? — Harshitha — 50 s, leave at 7:35

Step three: remove the server. Phase one needs the sum of the Stein vectors. Decentralised gradient methods stayed biased, and gradient tracking was exact but sent two hundred thousand scalars. A spanning tree is exact with under three thousand. For phase two, consensus and push-sum matched the server's regret but sent 140 to 285 million scalars. Event-triggered flooding came within 2 to 5 percent with seventy thousand. *[point at the red box]* An average only approaches the sum, so mixing has to run every round on the whole table. Relaying exact records delivers the sum itself, a few rounds late. From here on, everything relays.

15 min: 40 s, leave at 6:15. Drop the grey sentences.

12 Step 3, continued: a misleading win — Harshitha — 45 s, leave at 8:20

Two surprises. At one point the network beat the server: push-sum on a torus had lower regret. The cause was the adaptive stopping rule, because agents that disagreed kept exploring longer. So every later experiment fixes the phase-one length. Second, accelerated gossip is fragile. With 90 percent of links failing it was seven times worse, while flooding lost 27 percent, since flooding only needs each message to arrive eventually. After step three we were within 3 percent of the server with sixty thousand scalars.

15 min: skip; click straight to slide 13.

13 Step 4: test each claim before proving it — Harshitha — 45 s, leave at 9:05

Step four: before proving anything, we tested each claim. A server behaves like one agent with NT pulls, so the target rate is NT to the two-thirds. Losing the server costs 1.5 to 9 percent. Our sketched bound, N times D times the number of bins, was 60 to 150 times too big, and the measured cost barely grows with T. Also, if agents freeze different directions, regret goes up by about 30 percent, and one agreement pass fixes it. *[point at the red box]* That left three things to prove. Nischal will take the theory.

15 min: same.

14 Theory: setting of the proofs — Nischal — 30 s, leave at 9:35

Thanks. Our assumptions are Dey, Bhore and Ghosh's, with one made explicit: i.i.d. sub-Gaussian noise, which their Stein lemma already needs. Messages move one hop per round. The graph enters through the farness, F-prime: add up one agent's distances to everyone, and take the worst agent. It's at most N times D.

15 min: skip; Nischal starts at slide 15.

15 Theorem 5: no serverless algorithm beats NT to the two-thirds — Nischal — 45 s, leave at 10:20

First, what nobody can beat. Any N-agent algorithm, on any graph, even one where everyone sees everything after each round, has regret at least of order NT to the two-thirds. The proof is a simulation. One agent plays the N agents one after another, and holds back each round's rewards until all N have acted. The history has the same distribution, so the network's regret is one agent's regret over NT rounds. Then the single-agent lower bound applies. So the server's rate is the best a network can hope for.

15 min: 35 s, leave at 7:35. Drop the grey sentence.

16 Theorem 7: the graph does cost regret

Nischal 50 s, leave at 11:10

Theorem 7 says the graph itself costs regret. Information moves one hop per round, so whatever happens inside a light cone depends only on data inside it. Take disjoint cones of radius R. Each holds about R-squared agent-rounds and must learn on its own, so each pays its own two-thirds power. On a ring or a path, that adds a term N times the minimum of T and D, to the one-third. When T is small compared with root N D, no ring can match one agent with NT pulls. Our upper bound's graph term is linear in distance, so the two don't meet yet. That gap is open.

15 min: 35 s, leave at 8:10. Drop the grey sentences.

17 Step 5: the first proof breaks; epochs fix it

Nischal 50 s, leave at 12:00

Step five: our first proof broke. In the flood relay, an agent used its own unpublished rewards. A reward changes its next pulls, which changes when its trigger fires, which changes what everyone else sees. Each agent's view became a mix of N different prefixes, and the proof lost a factor of root N in the confidence width. Our first fix was synchronised epochs, where everyone acts on the same delayed table. That we could prove, with the right shape. *[point at the red box]* But every agent waits for the farthest one, and the extra regret was 3 to 6 times the relay's.

15 min: same.

18 Step 6: the algorithm we now run

Nischal 50 s, leave at 12:50

Step six is the algorithm we run now. Phase one: explore, keep a truncated Stein sum, flood it, and wait one delivery time. Every agent then holds all N sums and computes exactly the server's direction. Phase two: each round, add up the freshest published table from every agent, your own included once it's published, and pull the bin with the highest UCB index. If your unpublished pulls in some bin pass gamma times its count, flood your table. *[point at line 5]* The change from before is here: agents act only on published data. That's what makes the next theorem provable.

15 min: same.

19 Step 6: Theorem 27, regret of the algorithm we run

Nischal 50 s, leave at 13:40

Theorem 27 is the regret of that algorithm, for any gamma. Most terms are the server's: phase one, the UCB width, the bin width, the direction error. *[point at the last brace]* The only graph term is this one, and it doesn't grow with T. With gamma of order one over N, the rate is d N T to the two-thirds plus K times the farness, the sum of distances from the worst agent. That matches the lower bound while K F-prime stays below NT to the two-thirds. With a constant gamma, like the 0.5 we run, we pay a root of one plus N gamma but publish N times less.

15 min: same.

20 Why Theorem 27 holds: three ideas

Nischal 65 s, leave at 14:45

Three ideas. One: agents act only on published records, so a reward can't change when its own pull gets published. Every view is then one prefix of a single fixed order. That's a martingale, and the width is the server's, with no root N. Two: views nest. Whatever agent l saw by round s, agent i sees by s plus their distance. So l's hidden pulls are at most gamma times a count that i also sees, and that works for any gamma. Our first version needed gamma equal to one over 2N, which sends seven times the messages. Three: a pull needs an arm in that bin, and there are only K arms per round. So the pulls an agent can't yet see add up to about K F-prime in total, not per bin. That's why the graph term is free of T.

15 min: 50 s, leave at 11:30. Drop the grey sentences.

21 Evidence: the graph's cost has the proved shape

Nischal 40 s, leave at 15:25

Does the data look like the bound? Across graphs, our extra regret divided by K times the farness stays between 0.38 and 0.47, so it tracks the farness. Making T eight times longer raises it only 1.3 times. In K it grows like roughly root K, slower than the bound's linear K, so that factor is loose. And gamma at one over 2N does lower the excess, at about seven and a half times the scalars.

15 min: same.

22 Step 7: bin owners remove the rest of the gap

Nischal 50 s, leave at 16:15

Step seven: where does the rest of the gap come from? Far-apart agents explore the same empty bin before they hear about each other. Our fix is bin owners. A shared hash gives each bin an owner, so it costs no messages. Other agents leave an empty bin to its owner until they've seen it available ten times. With that rule, the excess over the server drops from between 118 and 643 to slightly negative on every graph, for 1.6 times the scalars. Two caveats. The rule also helps a server, and against that server we still pay 70 to 100. And we haven't proved it yet.

15 min: same.

23 Evidence: collaboration pays without a server

Nischal 20 s, leave at 16:35

Collaboration still pays without a server. Our relay keeps the server's N-to-the-0.43 scaling on well-connected graphs. At 32 agents the hypercube is within 7 percent of the server and the ring within 31.

15 min: skip; click straight to slide 24.

24 Theorem 29: communication

Nischal 30 s, leave at 17:05

Each message crosses each edge at most once, and a bin triggers a new publication only when its count grows by a factor of one plus gamma. So traffic is logarithmic in T, where sharing every round is linear. On a ring, gamma 0.5 sends 180 thousand scalars for 2 percent more regret; gamma one over 32 sends 1.4 million.

15 min: same.

25 Evidence: relaying vs mixing

Nischal 25 s, leave at 17:30

Our relay is within 3.4 percent of the server with 64 thousand scalars. Consensus and push-sum every round do as well on regret, sometimes slightly better, and send 2 to 4 thousand times more. Mixing pays in communication.

15 min: skip; click straight to slide 28 (skip 25 to 27).

26 Robustness: only the delivery time changes — Nischal — 25 s, leave at 17:55

A slow or failing link delays a record but never corrupts it. So directed, delayed or changing graphs only change the delivery time, and only the graph term grows. With 90 percent of links failing, our relay is 26 percent above the server, consensus 50 percent, and accelerated gossip seven times.

15 min: skip.

27 Theorem 33: agents must agree on the direction — Nischal — 25 s, leave at 18:20

If agents bin with slightly different directions, regret grows linearly in NT times the disagreement, so even a small disagreement hurts. One extra flood of the Stein sums makes everyone exact. On the ring that took the extra regret from 25 percent down to 8.

15 min: skip.

28 Results: prior papers' algorithms on our problem — Nischal — 30 s, leave at 18:50

Last, prior algorithms on our problem, each given ideal relaying. On the quadratic link we're best and the gap grows with T: kernel UCB has 3 times our regret by T of 4000, and linear or monotone methods 8 to 10 times. On the logistic link, LinUCB and ESTOR win, as they should, because they assume the monotonicity that holds there. Harshitha will wrap up.

15 min: same.

29 What the failures taught us — Harshitha — 25 s, leave at 19:15

This table is the talk on one page. Each row is something that failed, what it cost, and what we learned. Averaging estimates taught us to share exact sums. Consensus taught us to relay. Slow epochs taught us not to wait for the farthest agent. And duplicate first visits led to bin owners.

15 min: same.

30 Conclusion and future work — Harshitha — 40 s, leave at 19:55

Without a server we get the server's rate plus a graph cost that doesn't grow with T and is set by the farness. We prove the protocol we actually run, for any gamma, and the measurements have the predicted shape. The limits: our lower and upper graph terms don't meet, the bound is linear in K where we measure about root K, the data is synthetic, and the newest proofs are still being checked. Next, we want to prove the bin-owner rule and close the graph gap.

15 min: 30 s, leave at 14:55. Drop the grey sentence.

31 References — Harshitha — 0 s, leave at

[don't stop here; click straight to Thank you]

15 min: same.

32 Thank you — Harshitha — 5 s, leave at 20:00

Thank you. We're happy to take questions.

15 min: leave at 15:00.

If asked

Short answers to questions that came up while preparing. Say the first sentence; add the rest only if they push.

What does “5 to 20 times worse” on the timeline measure?
The error of the bin table, not regret. We start from a table that’s 10 rounds stale, sync once, and compare its error with the stale table’s. One-shot FedAvg ended 5 to 11 times worse, distillation up to 20. Regret moved only about 2 percent, because UCB pulls repair the table quickly.

Why do averaged counts break the UCB argument?
A mixed count isn’t a sample count. On a ring of 16 with our Metropolis weights, an agent with 10 pulls reports 53 after one round of consensus, so its bonus is more than twice too narrow. That overcount takes about a hundred rounds to fade, and new pulls keep arriving. You can’t write a confidence bound for a number that isn’t a count.

Is the gap to the server really zero with bin owners?
Against the plain server, yes, it’s slightly negative. But the owner rule also helps a server, and against that server we pay 70 to 100, nearly flat in the farness. And the rule isn’t proved yet.

Which lower bound depends on the graph?
Theorem 7: on rings and paths, N times min of T and D to the one-third, from light cones. Theorem 5 holds even with a server, so it doesn’t depend on the graph.

What do paired trials and 95 percent intervals mean here?
Every method runs on the same seeds, so they face identical contexts and noise. We report the mean plus or minus 1.96 standard errors. If the intervals don’t overlap, the difference isn’t seed luck.

Consensus gets the same regret. Why not just use it?
It sends 2 to 4 thousand times more, because an average only approaches the sum and has to run every round. It also depends on the spectral gap, which collapses when links fail: accelerated gossip was seven times worse at 90 percent failures.

Why is the bound linear in K when you measure root K?
Our counting charges every unseen agent-round K arms. We haven’t found a sharper argument yet; it’s listed as open.